MitM y TLS/SSL

© **Universidad de Palermo** | Prohibida la reproducción total o parcial de imágenes y textos.

UP
Universidad
de Palermo

Man in the Middle

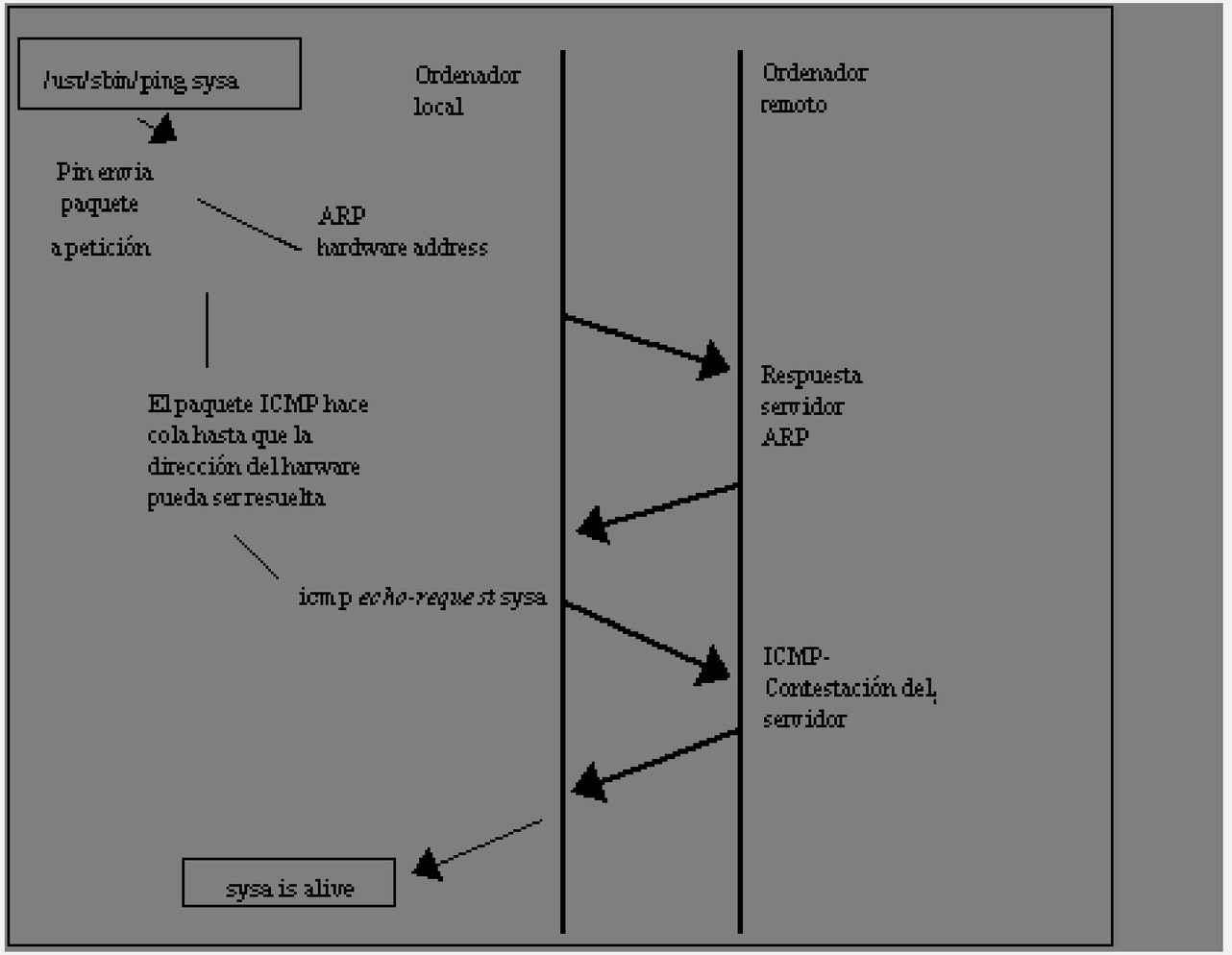
Conceptos iniciales

Cuestiones a tener en cuenta:

- Hub / Switch
- MAC Address
- IP
- Estado promiscuo
- ARP

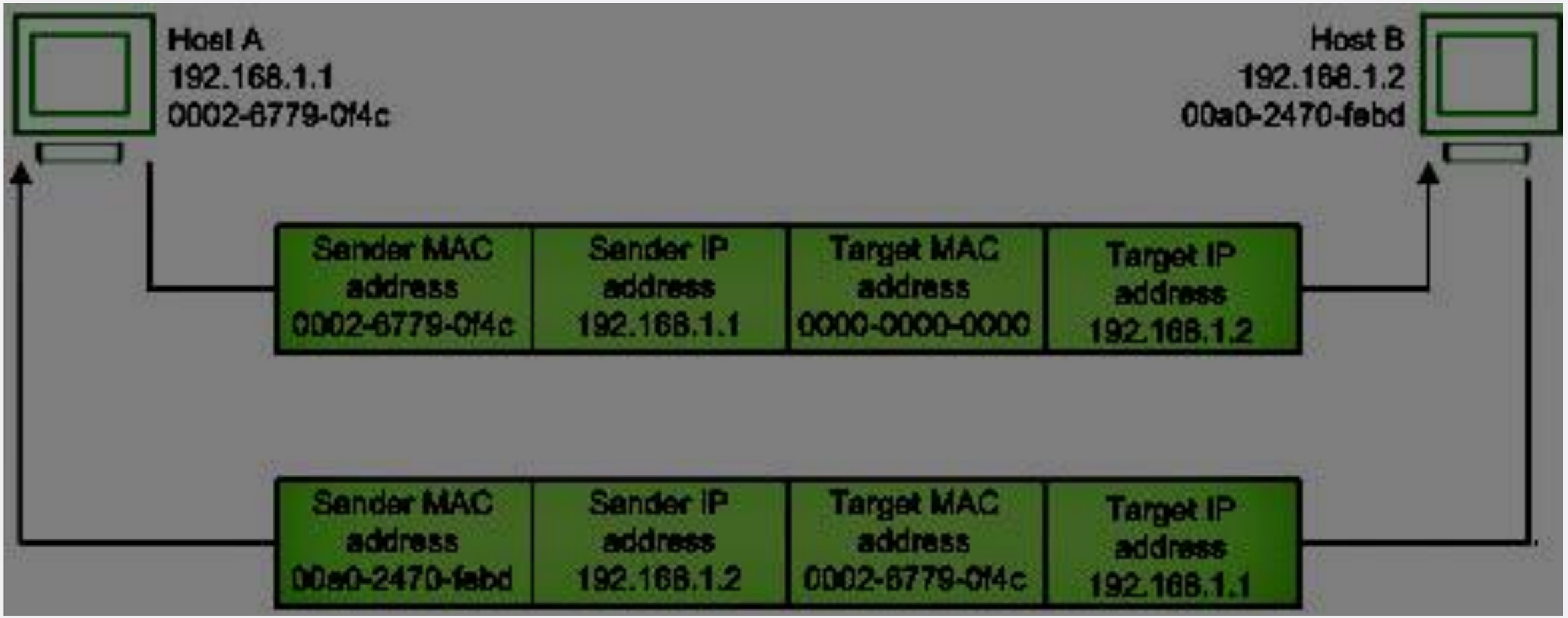
UP Universidad de Palermo ©

Address Resolution Protocol



Fuente: https://es.m.wikipedia.org/wiki/Protocolo_de_resoluci%C3%B3n_de_direcciones

Address Resolution Protocol



Fuente: https://www.geeksforgeeks.org/arp-reverse-arprarp-inverse-arp-inarp-proxy-arp-and-gratuitous-arp/

UP Universidad de Palermo ©

ARP Poisoning

El envenenamiento ARP es una técnica usada por atacantes en redes internas cuyo fin es obtener el tráfico de red circundante, aunque no esté destinado al sistema del propio intruso.

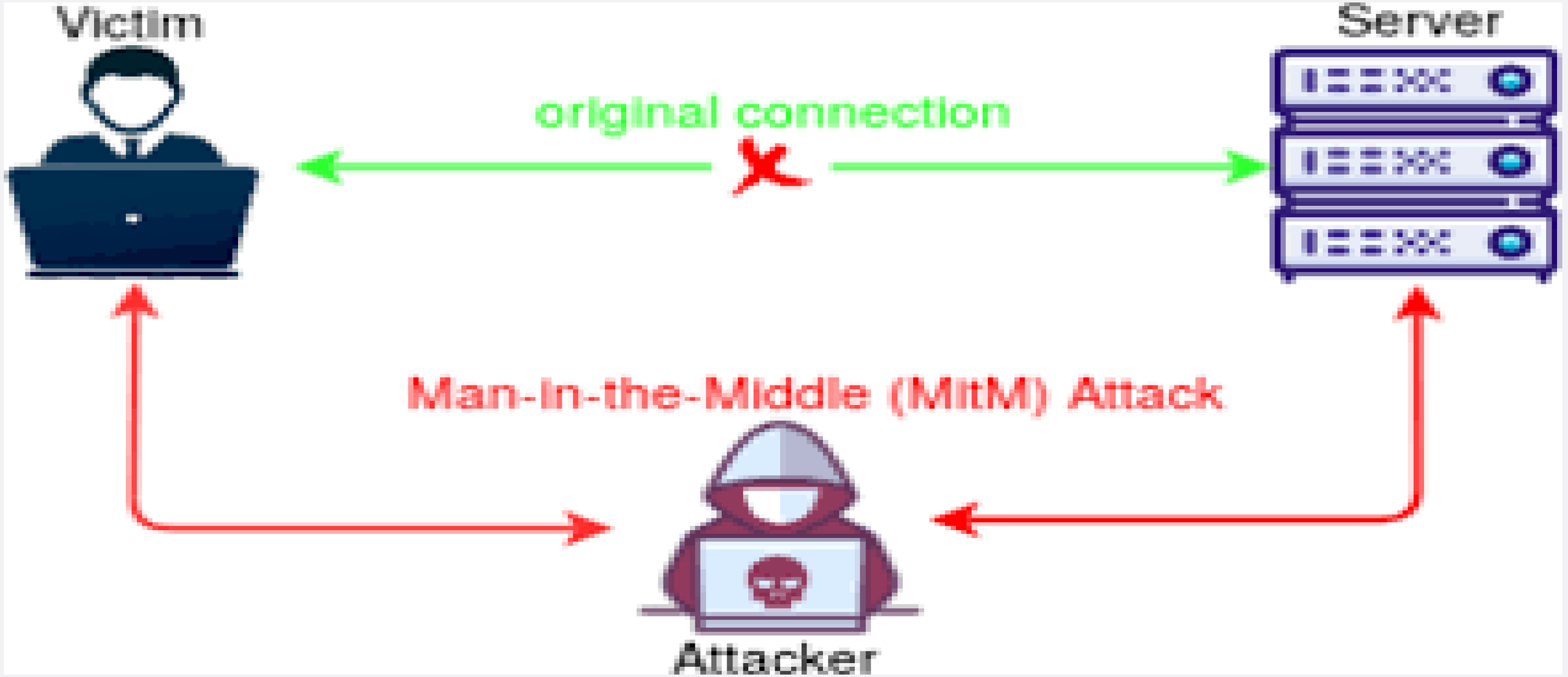
Con este método, el atacante puede conseguir desviar la información hacia su propia placa de red y de este modo conseguir información sensible, bloquearla o incluso modificarla y mostrar datos erróneos a las víctimas.

Esta técnica no se basa en una vulnerabilidad concreta que pueda llegar a desaparecer con el tiempo, sino que se basa en un fallo de diseño de las redes TCP/IP y, por tanto, es un método de ataque siempre válido y eficaz a menos que se tomen medidas específicas contra él.

ARP Poisoning funciona únicamente en Redes con IPv4.

UP Universidad de Palermo ©

¿Cómo funciona el ataque?



Fuente: https://link.springer.com/article/10.1007/s10207-021-00548-5

Tipos de ataque

- ARP DoS
- ARP Sniffing
- ARP Hijacking

UP Universidad de Palermo ©

Laboratorio

Laboratorio 1

Para este primer laboratorio necesitaremos:

VM Kali (Linux)

- arpspoof
- wireshark

VM 0wn3d (Windows 10)

- No tener aplicadas reglas de firewall.

Máquina host (Mi PC)

- Navegador web.

UP Universidad de Palermo ©

Laboratorio 1

Comandos a ejecutar en Kali:

arpspoof [IP 0wn3d] -t [IP Host] -r

Comando desde Host:

ping [IP 0wn3d]

Levantando Wireshark y filtrando por icmp ¿vemos el tráfico de consulta y respuesta?

UP Universidad de Palermo ©

Laboratorio 2

Comandos a ejecutar en Kali:

echo 1 > /proc/sys/net/ipv4/ip_forward

arpspoof [IP 0wn3d] -t [IP Host] -r

Comando desde Host:

ping [IP 0wn3d]

Levantando Wireshark y filtrando por icmp ¿vemos el tráfico de consulta y respuesta?

UP Universidad de Palermo ©

Laboratorio 3

Comandos a ejecutar en Kali:

echo 1 > /proc/sys/net/ipv4/ip_forward

arpspoof [IP 0wn3d] -t [IP Host] -r

Comando desde Host:

Iniciar la sesión en DVWA

Levantando Wireshark y filtrando por http, encontrar el usuario y la contraseña que viaja en texto plano.

UP Universidad de Palermo ©

Transport Layer Security

TLS

Es un **framework** que puede utilizarse en varios protocolos, lo más usual es utilizar este último sobre **HTTP** creando lo que se conoce como **HTTPS**.

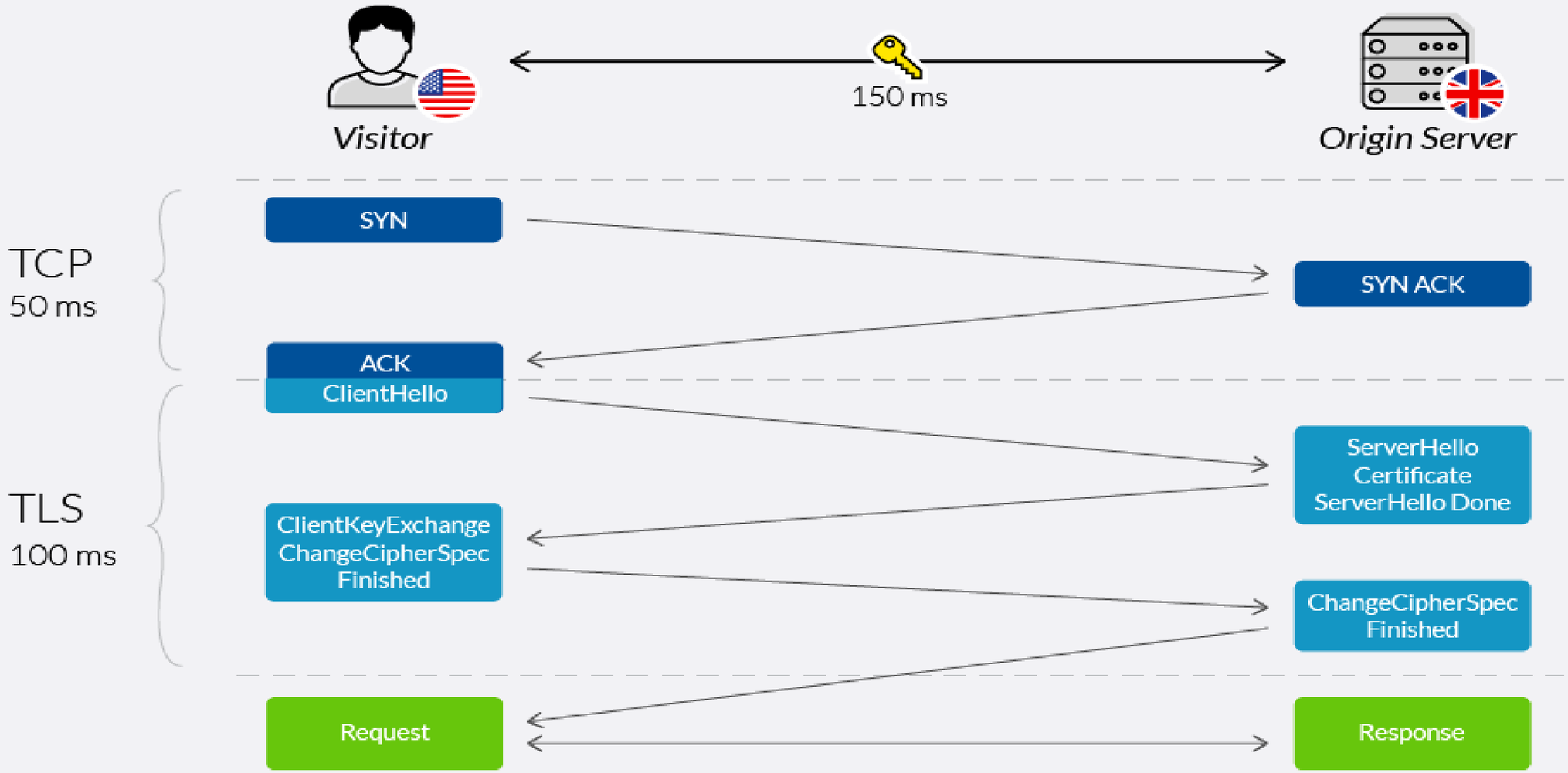
Proporciona autenticación y privacidad de la información entre extremos mediante el uso de criptografía.

Habitualmente, solo el servidor es autenticado (es decir, se garantiza su identidad) mientras que el cliente se mantiene sin autenticar.

Podría extenderse la autenticación por medio de certificados digitales también al cliente.

UP Universidad de Palermo ©

TLS Handshake



Fuente: https://www.imperva.com/learn/performance/ssl-tls/

Ataques contra SSL/TLS

BEAST (Browser Exploit Against SSL/TLS)

Esta vulnerabilidad fue descubierta en septiembre de 2011.

Fue presentada por Thai Duong y Juliano Rizzo y se basa en un programa JavaScript que junto con un sniffer de red desencripta las cookies de los clientes con la información privada de estos.

Esta vulnerabilidad se basa en un fallo encontrado en las suites de cifrado de TLS 1.0, concretamente en las suites de cifrado por bloque en modo CBC.

UP Universidad de Palermo ©

Ataques contra SSL/TLS

El problema se encuentra en que este tipo de cifrados estaban pensados para usar una vez para cifrar un archivo.

Sin embargo, en la capa SSL se usa para cifrar con la misma clave varios registros, que se irán enviando.

Por lo tanto, cada bloque siguiente tiene un residuo del anterior.

De esta forma si el atacante sabe que en uno de los mensajes se está enviando la contraseña, este sacará la contraseña inyectando un registro a la hora de cifrar.

Esta vulnerabilidad solo afecta a la versión 1.0 del protocolo TLS.

En cuanto a la gravedad de la vulnerabilidad, debemos mencionar que en principio parece difícil obtener una gran cantidad de información secreta de esta forma, pero es posible.

UP Universidad de Palermo ©

Ataques contra SSL/TLS

Aun así, después del tiempo que ha pasado desde que la vulnerabilidad fue descubierta, se han realizado multitud de cambios para que no pueda ser explotado.

Los navegadores de hoy día están parcheados para que no se pueda explotar, por todo ello, BEAST es poco grave.

La falla ha sido eliminada por parte de los navegadores web.

UP Universidad de Palermo ©

Ataques contra SSL/TLS

CRIME (Compression Ratio Info-leak Made Easy)

Esta vulnerabilidad se relaciona con la compresión de los datos antes de enviar el mensaje.

CRIME[14] (Compression Ratio Info-leak Made Easy) fue creada por las mismas personas que el ataque BEAST.

Dicho ataque se usa en conexiones donde se utilizan los protocolos **HTTPS** y **SPDY** que usan compresión y que permiten apropiarse de una sesión autenticada de un tercero.

UP Universidad de Palermo ©

Ataques contra SSL/TLS

Este ataque se basa en una combinación de inyección de texto plano y de fuga de información.

Para ello, el atacante inyecta pequeños trozos en los bloques a comprimir y posteriormente cifrar.

De esta forma, haciendo uso de varias conexiones y envío de mensajes (normalmente, un múltiplo de los bytes secretos a recuperar) es posible descifrar el mensaje original.

Esto sucede debido a que, a la hora de comprimir, si hay caracteres iguales juntos en el bloque, estos son comprimidos.

UP Universidad de Palermo ©

Ataques contra SSL/TLS

La falla ha sido eliminada por parte de los navegadores web.

Mientras que, por parte de los servidores, la única forma de mitigar este ataque es que el propio servidor priorice por un cifrado RC4, más que por un cifrado como AES o DES.

Aun así, se ha descubierto y verificado que el cifrado con RC4 es más débil de lo que se pensaba.

Por eso, es preferible mantener el uso de cifradores como DES y AES, puesto que BEAST solo se da en TLS 1.0, mientras que el uso de RC4 de forma insegura se puede dar en cualquier versión de TLS.

UP Universidad de Palermo ©

Laboratorio

POODLE (Padding Oracle On Downgraded Legacy Encryption)

La vulnerabilidad POODLE se hizo pública en octubre de 2014. Esta primera publicación hacía referencia a la vulnerabilidad encontrada en SSL 3.0 recogido en el CVE-2014-3566, mientras que dos meses después de la publicación de esta vulnerabilidad, en diciembre de 2014, se descubrió otro ataque que aparecía en el protocolo TLS recogido en el CVE-2014-8730.

Esta vulnerabilidad se aprovecha del protocolo de renegociación, intenta hacer que el servidor falle al establecer una conexión con el protocolo TLS y, de esta forma, el servidor aceptará una conexión de SSL 3.0.

Una vez conseguido esto, funciona de forma parecida al ataque BEAST, puesto que hace uso de algoritmos de cifrado por bloque en modo CBC, de esta forma se sabe que cada bloque debe de ser de un tamaño múltiplo de 8, Ataque CRIME 14 cifrado el bloque, el último byte contiene el tamaño de bytes de relleno usado para el bloque.

POODLE (Padding Oracle On Downgraded Legacy Encryption)

Así, un atacante puede directamente desechar los bytes de relleno y trabajar con el texto cifrado correspondiente al mensaje original.

Al igual que en el ataque BEAST, este hace uso de la inyección de datos en el texto, para así poder obtener la información secreta que contiene.

Gracias a este ataque se puede **obtener las cookies de sesión, los datos de autenticación y más información secreta.**

UP Universidad de Palermo ©

POODLE (Padding Oracle On Downgraded Legacy Encryption)

Para poder eliminar el ataque POODLE en SSL 3.0, la forma más sencilla es que el servidor deje de soportar dicho protocolo, pero no suele ser la mejor opción porque muchos de los clientes usan este protocolo.

Otra opción es implementar el **TLS_FALLBACK_SCSV**, que no deja hacer un **downgrade a SSL 3.0.**

Por último, se puede implementar la “división de registros anti-POODLE”, la cual hace imposible realizar el ataque.

Puesto que los registros se dividan en varias partes para que no se pueda usar la inyección de texto para sacar información.

SSLStrip y SSLSniff

UP Universidad de Palermo ©

SSLStrip

El funcionamiento de **SSLStrip** es simple, **reemplaza todas las peticiones HTTPS de una página web por HTTP** y luego hace un MITM (ataque Man in the Middle) entre el servidor y el cliente.

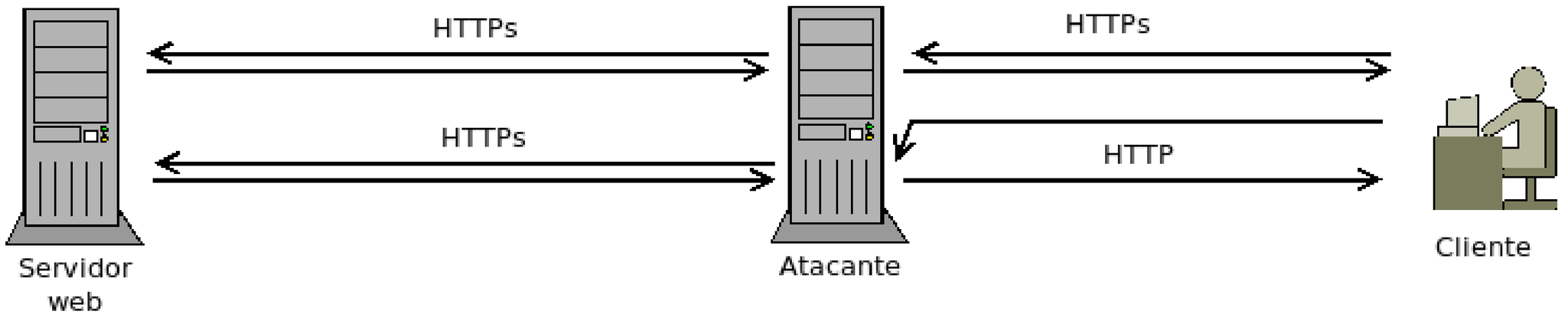
La idea es que la víctima y el agresor se comuniquen a través de HTTP, mientras que el atacante y el servidor se comunican a través de HTTPS con el certificado del servidor.

Por lo tanto, el atacante es capaz de ver todo el tráfico en texto plano de la víctima.

UP Universidad de Palermo ©

SSLStrip

SSLstrip es un ataque para capturar (y monitorizar en claro, claro está) conexiones HTTPs:



Fuente: https://sio2sio2.github.io/doc-linux/08.redes/99.ataques/02.tecnicas/04.mitm.html.

¿Cómo se implementa el sslstrip?

Independientemente de si se utiliza el sslstrip o una aplicación programada de forma similar, primero es necesario que el atacante active su **proxy entre el navegador y el servidor web.**

El software solo tiene la posibilidad de codificar direcciones URL modificadas mediante el SSL Stripping cuando puede interceptar o reenviar flujos de datos.

Entrada desapercibida del proxy en las opciones del navegador: a menudo y si tu sistema ha sido víctima de cibercriminales, el objetivo no es tu equipo, sino solo tu navegador.

El malware se asegura entonces de que un servidor proxy externo se integre automáticamente en los ajustes sin que el usuario lo note.

UP Universidad de Palermo ©

¿Cómo se implementa el sslstrip?

ARP o NDP spoofing: dentro de una subred, un atacante puede recurrir al llamado ARP spoofing (IPv4) o NDP Spoofing (IPv6) para mantener activo su proxy.

Ambos protocolos sirven para resolver direcciones IP en sus direcciones de hardware (también conocidas como direcciones MAC).

Valiéndose de los mensajes manipulados de estos protocolos, el atacante puede reemplazar las direcciones de hardware solicitadas con la de su propio sistema e interceptar los paquetes de datos transmitidos.

UP Universidad de Palermo ©

Laboratorio 1

sysctl -w=1 net.ipv4.ip_forward

iptables -t nat -F

iptables -t nat -A PREROUTING -p tcp - dport 80 -j REDIRECT - to-ports 8080

iptables -t nat -A PREROUTING -p tcp - dport 443- j REDIRECT - to-ports 8443

iptables -t nat -A PREROUTING -p tcp - dport 587- j REDIRECT - to-ports 8443

iptables -t nat -A PREROUTING -p tcp - dport 465- j REDIRECT - to-ports 8443

openssl genrsa -out ca1.key 4096

openssl req -new -x509 -days 1826 -key ca1.key -out ca2.crt

sudo sslsplit -D -l connections.log -j ./logdir -S logdir -k ca1.key -c ca2.crt ssl 0.0.0.0 8443

tcp 0.0.0.0 8080

UP Universidad de Palermo ©

¿Puede un usuario identificar un ataque sslstrip?

Los servidores y los navegadores no están en situación de detectar un sslstrip.

Ambas aplicaciones suponen que se están comunicando con el cliente real, por lo que no dudan en ningún momento de la integridad de los datos transmitidos.

Para los usuarios, la situación es bastante similar ya que, **a primera vista**, la visita a la página web deseada parece ser completamente normal.

En algunos casos excepcionales, las páginas manipuladas por SSL Stripping pueden reconocerse debido a pequeños detalles técnicos o de diseño.

Por lo tanto, si no se presenta un diseño especialmente defectuoso o no se producen retrasos considerables al cargar la página, no hay muchas pistas que dejen intuir que se ha burlado el **cifrado por SSL**.

UP Universidad de Palermo ©

¿Qué mecanismos de protección existen?

La dificultad de detectar páginas maliciosas hace que el SSL Stripping sea muy peligroso para los usuarios.

La razón de ser de los certificados de cifrado utilizados por los administradores web es garantizar **la seguridad y la fiabilidad**, eliminando las dudas de los usuarios a la hora de proporcionar sus datos privados.

UP Universidad de Palermo ©

¿Qué mecanismos de protección existen?

En principio, SSL también proporciona la protección necesaria, pues el hecho de que sea posible interceptar y leer paquetes de datos no es consecuencia de una brecha de seguridad en el protocolo, sino de la obstaculización del protocolo en sí mismo.

Para protegerse del sslstrip es recomendable que cada usuario refuerce las conexiones HTTPS cifradas mediante:

- **Entrada manual del URL:** una medida extremadamente tediosa pero eficaz es introducir manualmente la dirección URL HTTPS completa.
- **Extensión para el navegador:** existen diferentes extensiones del navegador que pueden ayudarte a acceder a las versiones cifradas, siempre y cuando las haya. La extensión HTTPS Everywehere utiliza listas de dominios y reglas para generar una conexión HTTPS para cada solicitud de una página web.
- **Guardar direcciones URL seguras como favoritos:** si utilizas un servicio web protegido con SSL (banca online, almacenamiento en la nube, etc.) frecuentemente, es recomendable que guardes su versión HTTPS como marcador y accedas a dicho servicio siempre a través de esta.

El requisito previo es que te encuentres en una red segura cuando la marques como favorita.

De lo contrario, correrás el riesgo de incluir un URL ya manipulado en la lista de favoritos.

UP Universidad de Palermo ©

SSLSniff

SSLSniff

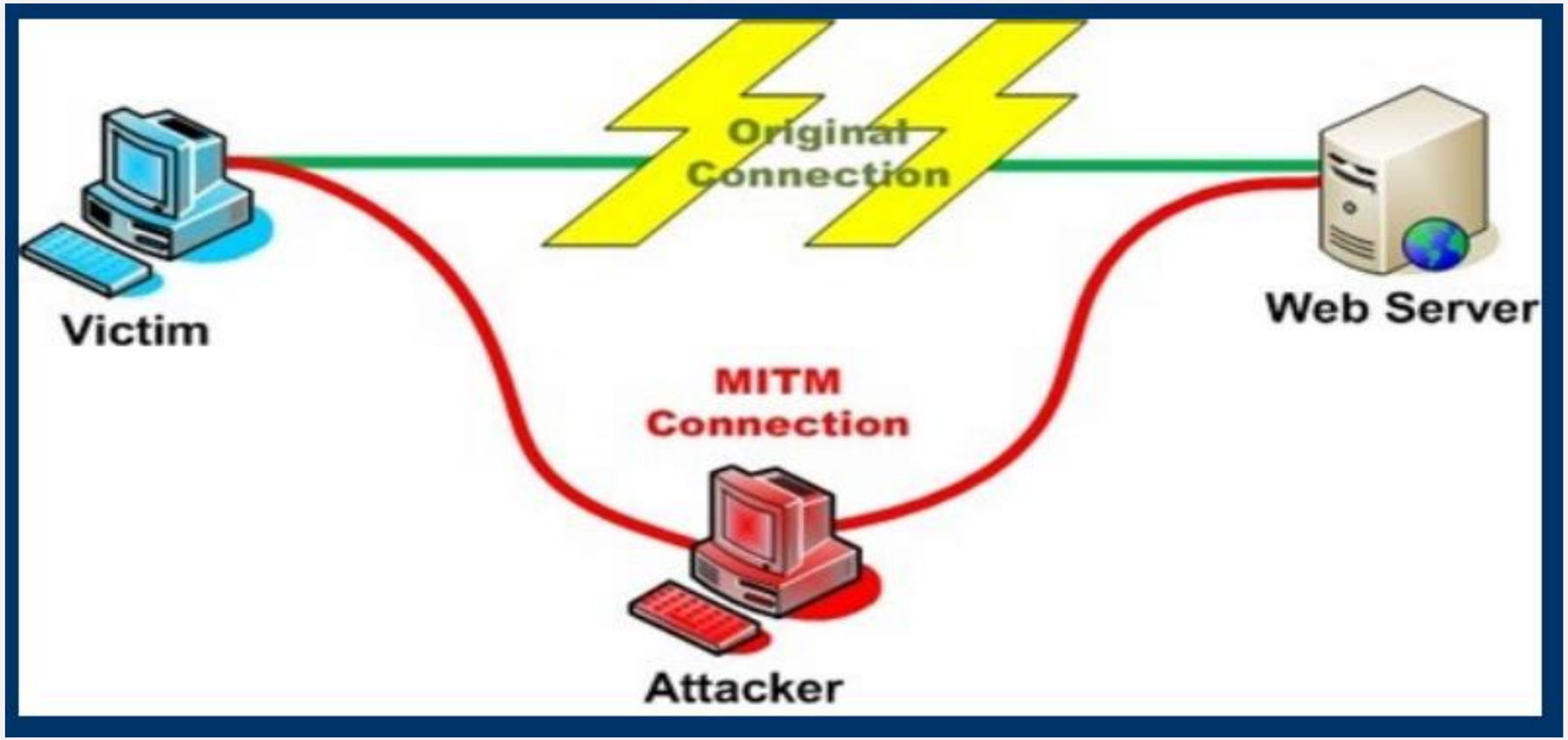
SSLSniff está diseñado para crear ataques man-in-the-middle (MITM) para conexiones SSL/TLS y genera dinámicamente certificados para los dominios a los que se accede sobre la marcha.

Los nuevos certificados se construyen en una cadena de certificados que está firmada por cualquier certificado que se proporcione.

SSLSniff también admite otros ataques como ataques de prefijo nulo o OCSP para lograr intercepciones silenciosas de conexiones cuando sea posible.

UP Universidad de Palermo ©

SSLSniff



Fuente: https://www.it.uu.se/edu/course/homepage/distrinfo/ht09/presentations/Group7.pdf

Canal covert y overt

Canal covert y overt

La diferencia entre un canal covert y overt se refiere a la forma en que se transmite la información y la medida en que es posible detectarla y protegerla.

Un canal overt es una comunicación que se realiza de manera abierta y pública, como por ejemplo una conversación por correo electrónico, un mensaje de texto o una publicación en una red social.

La información transmitida a través de un canal overt es fácilmente accesible y puede ser monitoreada o interceptada por terceros, incluyendo los ciberdelincuentes.

UP Universidad de Palermo ©

Canal covert y overt

Por otro lado, un canal covert se utiliza para transmitir información de manera secreta y sin ser detectado.

En ciberseguridad, los canales covert son utilizados por los ciberdelincuentes para ocultar información sensible o maliciosa que podría ser detectada o bloqueada por medidas de seguridad.

Los canales covert incluyen técnicas como el cifrado de mensajes, la estenografía (la ocultación de información dentro de archivos digitales) y el uso de protocolos y puertos no estándar para evadir la detección.

UP Universidad de Palermo ©

Canal covert y overt

En resumen, la diferencia entre un canal covert y overt en ciberseguridad es **la medida en que la información transmitida es detectable y accesible.**

Los **canales overt** son fáciles de detectar y monitorear, mientras que los **canales covert** son utilizados para ocultar información maliciosa y evitar la detección de medidas de seguridad.

Es importante que las organizaciones de ciberseguridad estén conscientes de estos canales y utilicen técnicas para detectar y protegerse contra las amenazas que se transmiten a través de ellos.

UP Universidad de Palermo ©

Canal covert y overt

Un ejemplo de un canal overt sería una comunicación a través de un correo electrónico empresarial en el que los empleados discuten un proyecto en curso.

Esta comunicación es visible y accesible para todos los participantes del correo electrónico y puede ser monitoreada por los administradores de la empresa.

Un ejemplo de un canal covert sería la esteganografía utilizada para ocultar información dentro de una imagen.

Un atacante podría utilizar esta técnica para enviar información maliciosa oculta dentro de una imagen, como un archivo de imagen de perfil en una red social, sin que nadie sospeche que la imagen contiene información oculta.

UP Universidad de Palermo ©

Canal covert y overt

Otro ejemplo de un canal covert podría ser el uso de un protocolo no estándar para evitar la detección de medidas de seguridad.

Por ejemplo, un atacante podría utilizar un puerto no estándar para transmitir información maliciosa a través de una red, evitando así la detección por parte de los sistemas de seguridad que monitorean los puertos estándar.

En ambos casos, es importante que las organizaciones de ciberseguridad estén conscientes de estas técnicas y utilicen medidas de seguridad para detectar y prevenir las amenazas que se transmiten a través de canales covert.

UP Universidad de Palermo ©

Troyano

Troyano

Un troyano, también conocido como caballo de Troya, es un tipo de software malicioso que se disfraza como un programa legítimo para engañar a los usuarios y lograr que lo instalen en sus dispositivos.

Una vez instalado, el troyano puede realizar una variedad de acciones maliciosas en el dispositivo de la víctima sin su conocimiento ni consentimiento.

Los troyanos pueden ser diseñados para robar información confidencial, como contraseñas, información bancaria y datos personales.

También pueden permitir que un atacante tome control remoto del dispositivo de la víctima y lo utilice para realizar acciones maliciosas, como el robo de información, el envío de spam o la realización de ataques de denegación de servicio (DDoS).

UP Universidad de Palermo ©

Troyano

Los troyanos pueden ser distribuidos a través de una variedad de medios, incluyendo correos electrónicos de phishing, descargas de software malicioso, sitios web infectados y mensajes de texto maliciosos.

Una vez que el troyano se instala en un dispositivo, puede ser difícil de detectar y eliminar, ya que a menudo está diseñado para ocultar sus actividades maliciosas.

Es importante tomar medidas de seguridad para protegerse contra los troyanos, como mantener actualizado el software de seguridad y no descargar software o archivos de fuentes no confiables.

También es importante estar atento a los correos electrónicos de phishing y otros mensajes sospechosos y no hacer clic en enlaces ni descargar archivos adjuntos de remitentes desconocidos.

UP Universidad de Palermo ©

Troyano

Hay varios tipos de troyanos, cada uno diseñado para realizar diferentes acciones maliciosas en un dispositivo.

Algunos de los tipos más comunes de troyanos incluyen:

- **Troyanos de acceso remoto:** estos troyanos permiten a un atacante tomar control remoto del dispositivo de la víctima, lo que les permite realizar una variedad de acciones maliciosas, como el robo de información, la instalación de software malicioso adicional y la realización de ataques de denegación de servicio (DDoS).
- **Troyanos bancarios:** este tipo de troyano está diseñado específicamente para robar información bancaria, como contraseñas y números de cuenta. Los troyanos bancarios a menudo se disfrazan como programas legítimos, como aplicaciones de banca móvil, para engañar a los usuarios para que los instalen.

UP Universidad de Palermo ©

Troyano

- **Troyanos de correo electrónico:** los troyanos de correo electrónico se propagan a través de correos electrónicos maliciosos que contienen archivos adjuntos infectados. Una vez que el usuario descarga y abre el archivo adjunto, el troyano se instala en su dispositivo.
- **Troyanos de descarga:** estos troyanos se descargan automáticamente en el dispositivo de la víctima cuando visita un sitio web infectado o descarga un archivo malicioso. Una vez instalado, el troyano puede realizar una variedad de acciones maliciosas, como el robo de información.

UP Universidad de Palermo ©

Troyano

- **Troyanos de puerta trasera:** los troyanos de puerta trasera son diseñados para proporcionar acceso a un dispositivo infectado por un atacante, incluso después de que se haya eliminado el troyano original. Estos troyanos a menudo se utilizan como parte de un ataque más grande y complejo.

Es importante protegerse contra los troyanos mediante la implementación de medidas de seguridad, como la actualización del software de seguridad, la descarga de software solo de fuentes confiables y la educación a los usuarios para que no abran correos electrónicos sospechosos o descarguen archivos de fuentes no confiables.

UP Universidad de Palermo ©

Botnet

UP Universidad de Palermo ©

Botnet

Una botnet es una red de dispositivos informáticos comprometidos y controlados de manera remota por un atacante.

Estos dispositivos comprometidos, también conocidos como "bots" o "zombies", a menudo se infectan con malware, como troyanos, que permiten a un atacante tomar el control remoto de los dispositivos infectados.

Una vez que los dispositivos están comprometidos, el atacante puede utilizarlos para realizar una variedad de acciones maliciosas sin el conocimiento ni el consentimiento de los usuarios legítimos de los dispositivos.

Por ejemplo, una botnet puede ser utilizada para enviar spam, distribuir malware, realizar ataques de denegación de servicio (DDoS) y realizar acciones de minería de criptomonedas.

UP Universidad de Palermo ©

Botnet

Los dispositivos que forman parte de una botnet a menudo son vulnerables debido a la falta de actualizaciones de seguridad o contraseñas débiles.

Los atacantes pueden utilizar herramientas automatizadas para escanear y encontrar dispositivos vulnerables, lo que les permite infectar fácilmente grandes cantidades de dispositivos para formar una botnet.

Es importante protegerse contra las botnets mediante la implementación de medidas de seguridad, como mantener actualizado el software de seguridad, cambiar las contraseñas por defecto y no descargar software de fuentes no confiables.

También es importante estar atento a los signos de infección, como el bajo rendimiento del dispositivo, los cambios en la configuración y el aumento del tráfico de red, y tomar medidas inmediatas para solucionar el problema si se sospecha de una infección.

UP Universidad de Palermo ©

Botnet

Uno de los ejemplos más conocidos es la **botnet Mirai**, que en 2016 fue utilizada para realizar un ataque DDoS contra el proveedor de servicios de DNS Dyn, lo que resultó en la interrupción del acceso a sitios web de alto perfil como Twitter, Reddit y Netflix.

La botnet Mirai fue creada a partir de dispositivos IoT (Internet de las cosas) comprometidos, como cámaras de seguridad y enrutadores, y fue utilizada para lanzar ataques masivos.

Otro ejemplo es la botnet Emotet, que ha estado activa desde 2014 y ha sido utilizada para distribuir malware y robar credenciales.

Emotet a menudo se propaga a través de correos electrónicos de phishing y ha sido responsable de grandes campañas de distribución de malware.

UP Universidad de Palermo ©



© Universidad de Palermo
Prohibida la reproducción total o parcial de imágenes y textos.